Machine Learning & Deep Learning — Comprehensive Q&A Reference

A dense, interview-style reference spanning classical ML through modern deep learning. Organized by topic; each entry is a self-contained Q&A.

Table of Contents

---

1. Core ML Fundamentals

Q1. What is the bias-variance tradeoff? Bias is error from overly simplistic assumptions (underfitting); variance is error from sensitivity to training data fluctuations (overfitting). Total expected error = Bias² + Variance + Irreducible noise. Increasing model capacity typically lowers bias but raises variance.

Q2. What's the difference between overfitting and underfitting, and how do you detect each? Overfitting: low training error, high validation error (model memorizes noise). Underfitting:

high error on both (model too simple). Detected via train/val loss curves — a widening gap signals overfitting; both curves plateauing high signals underfitting.

Q3. What is regularization? Compare L1 vs L2. Regularization adds a penalty to the loss to discourage overly complex models. L1 (Lasso) adds $\Sigma|w|$, producing sparse weights (feature selection). L2 (Ridge) adds Σw^2, shrinking weights smoothly without zeroing them. Elastic Net combines both.

Q4. Why does L1 induce sparsity but L2 doesn't? Geometrically, L1's constraint region is a diamond with corners on the axes, so the loss contours are likely to intersect at a corner (a zero coordinate). L2's constraint region is a smooth sphere, so intersections rarely land exactly on an axis.

Q5. What is cross-validation and why use k-fold? It estimates generalization performance by splitting data into k folds, training on k-1 and validating on the remaining fold, rotating through all folds. It reduces variance in the performance estimate compared to a single train/val split, especially with limited data.

Q6. Explain precision, recall, F1, and when each matters. Precision = TP/(TP+FP) — how many predicted positives are correct. Recall = TP/(TP+FN) — how many actual positives were found. F1 is their harmonic mean. Use precision-focused metrics when false positives are costly (spam filters); recall-focused when false negatives are costly (cancer screening).

Q7. What is ROC-AUC and how does it differ from PR-AUC? ROC plots TPR vs FPR across thresholds; AUC is the probability a random positive scores higher than a random negative. PR-AUC (precision vs recall) is more informative than ROC-AUC on highly imbalanced datasets, where ROC can look overly optimistic.

Q8. What is the difference between MLE and MAP estimation? MLE maximizes the likelihood of data given parameters, $P(D|\theta)$. MAP incorporates a prior, maximizing $P(\theta|D) \propto P(D|\theta)P(\theta)$. L2 regularization is equivalent to MAP with a Gaussian prior on weights; L1 corresponds to a Laplace prior.

Q9. Explain gradient descent and its major variants. Batch GD uses the full dataset per update (stable but slow). SGD uses one sample at a time (noisy but fast, can escape local minima). Mini-batch GD balances both. Momentum accumulates a velocity term to dampen oscillations; RMSProp/Adam adapt per-parameter learning rates using gradient history.

Q10. Derive/explain the Adam optimizer. Adam maintains exponential moving averages of the gradient (first moment, m) and squared gradient (second moment, v), bias-corrects them, then updates $\theta \leftarrow \theta - \eta \cdot \hat{m}/(\sqrt{\hat{v}} + \varepsilon)$. It combines momentum with per-parameter adaptive learning rates (from RMSProp).

Q11. What is the difference between parametric and non-parametric models? Parametric models (linear regression, neural nets) assume a fixed functional form with a fixed number of parameters regardless of data size. Non-parametric models (KNN, kernel methods, decision trees) grow in complexity with the data and make fewer structural assumptions.

Q12. What is the curse of dimensionality? As dimensionality grows, data becomes sparse in the feature space, distances between points become less discriminative, and the volume needed to maintain sample density grows exponentially — hurting distance-based methods like KNN and

increasing overfitting risk.

Q13. Cross-entropy loss vs MSE for classification — why prefer cross-entropy? Cross-entropy directly penalizes the predicted probability of the true class and produces well-behaved (non-vanishing) gradients when combined with softmax/sigmoid. MSE with a sigmoid output saturates and gives very small gradients when predictions are confidently wrong, slowing learning.

Q14. What is class imbalance and how do you handle it? When one class dominates, models bias toward predicting it. Fixes: resampling (oversample minority/undersample majority), SMOTE, class-weighted loss, focal loss, or using metrics like PR-AUC/F1 instead of accuracy.

---

2. Linear & Logistic Regression

Q15. State the assumptions of linear regression. Linearity between X and y, independence of errors, homoscedasticity (constant error variance), normally distributed residuals, and no multicollinearity among predictors.

Q16. Derive the closed-form solution for linear regression. Minimizing $||y - Xw||^2$ gives the normal equation $w = (X^TX)^{-1}X^Ty$. This requires X^TX to be invertible; when it's not (multicollinearity), ridge regression adds λI: $w = (X^TX + \lambda I)^{-1}X^Ty$.

Q17. Why is logistic regression called "regression" if it's used for classification? It regresses the log-odds (logit) of the positive class as a linear function of inputs: $\log(p/(1-p)) = w^Tx$. The output is then squashed through a sigmoid to get a probability, and thresholded for classification.

Q18. What loss does logistic regression minimize, and why? Binary cross-entropy, derived from maximizing the Bernoulli likelihood of the data. It's convex in the logistic regression setting, guaranteeing a unique global optimum under gradient descent.

Q19. What is multicollinearity and why is it a problem? When predictors are highly correlated, coefficient estimates become unstable and their standard errors inflate, making individual coefficients hard to interpret (though prediction accuracy may be unaffected). Detected via Variance Inflation Factor (VIF); fixed via regularization or dropping redundant features.

Q20. Ridge vs Lasso vs Elastic Net — when to use which? Ridge: many small/medium correlated predictors, want to shrink but keep all. Lasso: want automatic feature selection / sparse solutions. Elastic Net: many correlated features where Lasso alone is unstable — combines L1's sparsity with L2's stability.

---

3. Trees & Ensembles

Q21. How does a decision tree choose splits? It picks the feature/threshold that maximizes information gain (reduction in entropy) or minimizes Gini impurity at each node, evaluated greedily at every split rather than globally optimized.

Q22. Define entropy and Gini impurity. Entropy = $-\Sigma\, p_i \log_2(p_i)$. Gini = $1 - \Sigma\, p_i^2$. Both measure node "impurity" (0 = pure node); Gini is cheaper to compute (no log) and is CART's default, while entropy is used in ID3/C4.5.

Q23. Why do decision trees overfit, and how is this controlled? Unconstrained trees keep splitting until nodes are pure, memorizing training data (deep trees = high variance). Controlled via max depth, min samples per leaf, minimum impurity decrease, or post-pruning (cost-complexity pruning).

Q24. What is bagging, and how does Random Forest use it? Bagging (Bootstrap Aggregating) trains multiple models on bootstrap-resampled subsets of data and averages/votes predictions to reduce variance. Random Forest adds feature subsampling at each split (not just row sampling), decorrelating trees further and improving generalization.

Q25. How does boosting differ from bagging? Bagging trains models independently in parallel to reduce variance. Boosting trains models sequentially, each new model focusing on the errors of the previous ensemble, primarily reducing bias (and some variance).

Q26. Explain AdaBoost. AdaBoost trains weak learners sequentially, upweighting misclassified samples after each round so the next learner focuses on hard examples. Final prediction is a weighted vote, with each learner's weight (alpha) proportional to its accuracy.

Q27. Explain Gradient Boosting (GBM). Each new tree is fit to the negative gradient (residual) of the loss function with respect to the current ensemble's predictions, effectively performing gradient descent in function space. Predictions are added with a shrinkage/learning-rate factor to control overfitting.

Q28. What does XGBoost add over vanilla gradient boosting? A regularized objective (penalizing tree complexity — number of leaves and leaf weights), second-order (Newton) gradient information for more accurate splits, built-in handling of missing values, column subsampling, and highly optimized parallel/histogram-based split finding.

Q29. Random Forest vs Gradient Boosted Trees — key tradeoffs? Random Forest: trains in parallel, robust to overfitting, less tuning-sensitive, generally slightly lower accuracy ceiling. GBM/XGBoost: trains sequentially, often higher accuracy with careful tuning, but more prone to overfitting and slower to train.

Q30. What is out-of-bag (OOB) error? In bagging, each tree is trained on ~63% of samples (bootstrap sampling with replacement); the remaining ~37% ("out-of-bag") can be used to estimate validation error without a separate held-out set.

Q31. How do trees handle categorical and missing features natively? Many tree implementations can split categorical features directly (grouping categories) and route missing values down a default branch learned during training (as in XGBoost), avoiding the need for one-hot encoding or imputation.

---

4. SVM & Classical Classifiers

Q32. What is the intuition behind SVM? SVM finds the hyperplane that maximizes the margin (distance) between the closest points of each class (support vectors). A larger margin generally improves generalization on unseen data.

Q33. Explain the kernel trick. Instead of explicitly mapping data to a higher-dimensional space $\varphi(x)$ to make it linearly separable, kernels compute the inner product $K(x_i,x_j) = \varphi(x_i)\cdot\varphi(x_j)$ directly,

avoiding the (potentially infinite-dimensional) explicit transformation. Common kernels: linear, polynomial, RBF (Gaussian).

Q34. What does the soft-margin / C parameter control in SVM? C balances margin width against misclassification: large C penalizes margin violations heavily (narrower margin, less tolerance for errors, risk of overfitting); small C allows more violations (wider margin, more regularized, risk of underfitting).

Q35. What is the primal vs dual formulation of SVM, and why use the dual? The primal directly optimizes over weights w and bias b. The dual (via Lagrangian) optimizes over per-sample multipliers α_i, expressing the decision boundary purely in terms of dot products between samples — which is what enables the kernel trick.

Q36. How does KNN work, and what's the effect of k? KNN classifies a point by majority vote (or averages, for regression) among its k nearest neighbors by some distance metric. Small k → low bias, high variance (sensitive to noise). Large k → smoother decision boundary, higher bias, lower variance.

Q37. Why is feature scaling important for KNN and SVM but not for trees? KNN/SVM rely on distances or dot products, so features with larger numeric ranges dominate unless scaled. Trees split on threshold comparisons per feature independently, so the scale of a feature doesn't affect the split point chosen.

Q38. Explain Naive Bayes and the "naive" assumption. It applies Bayes' theorem to classify by computing $P(\text{class}|\text{features}) \propto P(\text{class}) \cdot \Pi\, P(\text{feature}_i|\text{class})$, assuming features are conditionally independent given the class — an assumption that's often violated but works well in practice, especially for text classification.

---

5. Unsupervised Learning & Dimensionality Reduction

Q39. Explain PCA step by step.

1. Center (and optionally standardize) the data. 2) Compute the covariance matrix. 3) Find its eigenvectors/eigenvalues (or use SVD on the data matrix directly). 4) Sort eigenvectors by descending eigenvalue. 5) Project data onto the top-k eigenvectors (principal components), which capture maximum variance.

Q40. What do PCA's eigenvalues and eigenvectors represent? Eigenvectors of the covariance matrix are the principal component directions (orthogonal axes of maximum variance); eigenvalues represent the amount of variance captured along each corresponding direction.

Q41. How do you choose the number of principal components to keep? Common approaches: keep components explaining a target cumulative variance (e.g., 95%), use a scree plot and look for an "elbow," or cross-validate downstream task performance at different k.

Q42. PCA via SVD — why is this preferred in practice? PCA can be computed by taking the SVD of the centered data matrix $X = U\Sigma V^T$; the columns of V are the principal components, and singular values relate to eigenvalues by $\sigma_i^2 = (n-1)\lambda_i$. SVD avoids explicitly forming the covariance matrix, which is more numerically stable for high-dimensional data.

Q43. What are the limitations of PCA? It only captures linear relationships, is sensitive to feature scaling, is sensitive to outliers, and the resulting components are often hard to interpret since they're linear combinations of all original features.

Q44. PCA vs LDA — what's the key difference? PCA is unsupervised and finds directions of maximum variance without regard to class labels. LDA (Linear Discriminant Analysis) is supervised and finds directions that maximize between-class separability relative to within-class scatter — better for classification-oriented dimensionality reduction.

Q45. Explain t-SNE and its main limitation. t-SNE converts pairwise distances into probabilities (Gaussian in high-D, Student-t in low-D) and minimizes KL divergence between the two distributions, preserving local neighborhood structure for visualization. Limitation: it doesn't preserve global structure/distances well, is sensitive to the perplexity hyperparameter, and is non-deterministic and non-parametric (can't easily embed new points).

Q46. How does UMAP differ from t-SNE? UMAP is built on manifold learning and topological data analysis, generally preserves more global structure than t-SNE, runs faster on large datasets, and (unlike vanilla t-SNE) supports transforming new/unseen points after fitting.

Q47. Explain K-means clustering and its objective. K-means minimizes within-cluster sum of squared distances (inertia) by iterating: (1) assign each point to its nearest centroid, (2) recompute centroids as the mean of assigned points, until convergence. It's sensitive to initialization (hence k-means++) and assumes roughly spherical, similarly-sized clusters.

Q48. How do you choose k in K-means? Elbow method (plot inertia vs k, look for the bend), silhouette score (measures how well-separated clusters are), or domain knowledge/downstream task validation.

Q49. Explain DBSCAN and how it differs from K-means. DBSCAN groups points that are densely packed (within ε distance and minPts neighbors), labeling sparse points as noise/outliers. Unlike K-means, it doesn't require specifying the number of clusters upfront and can find arbitrarily shaped clusters, but struggles with varying-density clusters.

Q50. What is hierarchical clustering (agglomerative)? It starts with each point as its own cluster and iteratively merges the closest pair of clusters (by a linkage criterion — single, complete, average, or Ward) until one cluster remains, producing a dendrogram that can be cut at any level for a desired number of clusters.

---

6. Neural Network Foundations

Q51. What is backpropagation, in one sentence? It's the application of the chain rule to efficiently compute the gradient of the loss with respect to every parameter in the network by propagating error signals backward from the output layer to the input layer.

Q52. Why do we need non-linear activation functions? Without them, stacking linear layers collapses to a single linear transformation regardless of depth (a composition of linear functions is linear), so the network couldn't represent non-linear decision boundaries or functions.

Q53. Compare ReLU, sigmoid, tanh, and GELU. Sigmoid squashes to (0,1) but saturates and causes vanishing gradients; tanh is zero-centered but has the same saturation issue. ReLU

(max(0,x)) is cheap and avoids saturation for positive inputs but can "die" (zero gradient) for negative inputs. GELU smoothly weights inputs by their probability under a Gaussian CDF, used in BERT/GPT/ViT for smoother gradients than ReLU.

Q54. What is the dying ReLU problem, and how is it mitigated? If a ReLU neuron's weights update such that its input is always negative, its gradient becomes permanently zero and it stops learning. Mitigations: Leaky ReLU / PReLU (small negative slope), ELU, GELU, or careful initialization and lower learning rates.

Q55. What causes vanishing and exploding gradients? In deep networks, gradients are products of many layer-wise Jacobians via the chain rule. If these terms are consistently <1 (e.g., saturating activations), gradients shrink exponentially with depth (vanishing); if consistently >1, they grow exponentially (exploding).

Q56. How do we mitigate vanishing/exploding gradients? Careful weight initialization (Xavier/He), non-saturating activations (ReLU family), normalization layers (BatchNorm/LayerNorm), residual/skip connections, gradient clipping (for exploding gradients), and architectures like LSTM/GRU designed to preserve gradient flow over time.

Q57. Explain Xavier vs He initialization. Xavier (Glorot) initialization scales weights by $1/\sqrt{(\text{fan\_in})}$ (or averaged fan_in/fan_out) to keep activation variance stable across layers, derived assuming linear/tanh activations. He initialization scales by $\sqrt{(2/\text{fan\_in})}$, accounting for ReLU zeroing out half the inputs, which would otherwise halve the variance each layer.

Q58. What is dropout and why does it work? Dropout randomly zeroes a fraction of activations during training, forcing the network to not rely on any single neuron and effectively training an ensemble of thinned sub-networks that are implicitly averaged at test time (with activations scaled accordingly).

Q59. Why is dropout usually disabled/scaled at inference? At test time we want a deterministic, full-capacity forward pass. "Inverted dropout" scales activations by $1/(1-p)$ during training so that no rescaling is needed at inference, keeping expected activation magnitude consistent between train and test.

Q60. What's the difference between an epoch, a batch, and an iteration? An epoch is one full pass through the training dataset. A batch is a subset of samples used for one gradient update. An iteration is one such update; the number of iterations per epoch = dataset_size / batch_size.

Q61. Why use mini-batches instead of the full dataset or single samples? Mini-batches balance the gradient estimate's stability (better than single-sample SGD) against computational efficiency and memory (better than full-batch GD), and they leverage parallel hardware (GPU) efficiently while retaining some beneficial noise that helps escape sharp local minima.

Q62. What is the universal approximation theorem? It states that a feedforward network with a single hidden layer of sufficient (possibly very large) width and a non-linear activation can approximate any continuous function on a compact domain to arbitrary precision — though it says nothing about how easy such a network is to train or how many neurons are needed practically.

Q63. Why do deep networks often generalize despite having far more parameters than training samples? Empirically, gradient descent on over-parameterized networks tends to find flatter minima and is implicitly regularized (via SGD noise, architecture inductive biases, and early

stopping); this is an active research area (e.g., double descent phenomenon) rather than a fully settled theoretical question.

---

7. Normalization Techniques

Q64. Explain Batch Normalization — formula and purpose. For a mini-batch, BN normalizes each feature: $\hat{x} = (x - \mu_{batch})/\sqrt{(\sigma^2_{batch} + \varepsilon)}$, then applies a learnable affine transform $y = \gamma\hat{x} + \beta$. It reduces internal covariate shift, allows higher learning rates, and acts as a mild regularizer, but its statistics depend on batch composition.

Q65. Why does BatchNorm struggle with small batch sizes or RNNs? With small batches, the batch statistics (mean/variance) become noisy, unstable estimates, hurting training. For RNNs, statistics would need to be computed per time step, and batches often have variable sequence lengths, making consistent normalization awkward.

Q66. Explain Layer Normalization and how it differs from BatchNorm. LayerNorm normalizes across the feature dimension for each individual sample (independent of other samples in the batch), rather than across the batch dimension per feature. This makes it batch-size independent, well-suited to RNNs and Transformers where batch statistics are unreliable or sequence lengths vary.

Q67. Compare BatchNorm, LayerNorm, InstanceNorm, and GroupNorm. BatchNorm: normalize per-channel across the batch (and spatial dims for CNNs). LayerNorm: normalize across all features for each sample. InstanceNorm: normalize per-channel, per-sample (popular in style transfer). GroupNorm: normalize within groups of channels, per-sample — a middle ground that works well with small batch sizes, common in detection/segmentation.

Q68. Why do Transformers use LayerNorm instead of BatchNorm? Transformers process variable-length sequences and are often trained/inferred with small or single-example batches (especially at inference/autoregressive decoding), where batch statistics are unreliable; LayerNorm's per-sample normalization avoids this dependency entirely.

Q69. What is RMSNorm, and why do modern LLMs (e.g., LLaMA) use it over LayerNorm? RMSNorm normalizes only by the root-mean-square of activations (skipping mean-centering): $\hat{x} = x/\sqrt{(\text{mean}(x^2)+\varepsilon)}$, then scales by a learnable γ (no shift term β). It's computationally cheaper and empirically works comparably to LayerNorm while simplifying the operation.

Q70. Pre-LN vs Post-LN Transformer blocks — what's the difference and why does it matter? Post-LN (original Transformer) applies LayerNorm after the residual addition; Pre-LN applies it before the sublayer (inside the residual branch). Pre-LN yields more stable gradients and training at greater depth (avoiding the need for careful warmup), which is why most modern large Transformers use Pre-LN.

Q71. What problem does normalization solve regarding internal covariate shift? As parameters update during training, the distribution of each layer's inputs shifts, forcing subsequent layers to continually re-adapt, which slows convergence. Normalization stabilizes these input distributions layer-to-layer (though the precise mechanism of why BN helps is still debated — some argue it primarily smooths the loss landscape rather than reducing covariate shift per se).

---

8. CNNs & Classic Architectures

Q72. Explain the convolution operation and its key hyperparameters. A convolution slides a learnable kernel/filter over the input, computing a weighted sum (dot product) at each position to produce a feature map. Key hyperparameters: kernel size, stride (step size), padding (border handling — "same" vs "valid"), and dilation (spacing between kernel elements).

Q73. Why are CNNs more parameter-efficient than fully connected layers for images? Convolutions exploit weight sharing (the same filter is applied across all spatial locations) and local connectivity (each output depends only on a small receptive field), drastically reducing parameters versus a dense layer connecting every input pixel to every output unit, while encoding useful inductive biases (translation invariance).

Q74. What is the receptive field, and how does it grow with depth? The receptive field is the region of the input that influences a given output unit. It grows with each additional convolutional/pooling layer (roughly additively with kernel size, and multiplicatively with stride/pooling), which is why deeper networks can capture larger-context/more global patterns.

Q75. What is pooling, and why use it (max vs average)? Pooling downsamples feature maps, reducing spatial resolution and computation while providing translation invariance. Max pooling keeps the strongest activation in each window (good for detecting presence of a feature); average pooling smooths and retains more background context.

Q76. Describe AlexNet's key contributions. AlexNet (2012) popularized deep CNNs by combining ReLU activations (faster training than tanh/sigmoid), dropout for regularization, data augmentation, local response normalization, and GPU-based training on large-scale ImageNet data — dramatically beating prior approaches and kickstarting the deep learning era in vision.

Q77. What did VGGNet contribute? VGG showed that stacking many small 3×3 convolutions (instead of larger kernels) increases effective receptive field while using fewer parameters and adding more non-linearities, and demonstrated that simply increasing depth (with a uniform, simple architecture) improves accuracy.

Q78. Why do skip/residual connections in ResNet help train very deep networks? Instead of learning a direct mapping $H(x)$, each block learns a residual $F(x) = H(x) - x$, and the output is $F(x) + x$. This identity shortcut lets gradients flow directly backward without attenuation through many layers (mitigating vanishing gradients) and makes it easy for a block to learn an identity function if extra depth isn't needed, avoiding degradation as networks get deeper.

Q79. What is the "degradation problem" that ResNet solves (distinct from overfitting)? Prior to ResNet, simply stacking more layers on plain (non-residual) networks caused training error to increase, not just test error — indicating an optimization difficulty (harder to learn identity-like mappings through many stacked non-linear layers), not overfitting. Residual connections directly address this.

Q80. Explain the Inception/GoogLeNet module. Instead of choosing a single kernel size, an Inception block applies multiple convolutions (1×1, 3×3, 5×5) and pooling in parallel on the same input and concatenates their outputs, letting the network learn which scale of features matters. 1×1 convolutions are used beforehand as "bottlenecks" to reduce channel depth and computational cost.

Q81. What does a 1×1 convolution do, and why is it useful? It applies a linear (plus non-

linearity) transformation across channels at each spatial location independently, commonly used to reduce/increase channel dimensionality cheaply (bottlenecking) or to add non-linearity without affecting spatial resolution.

Q82. What is depthwise separable convolution (used in MobileNet)? It factorizes a standard convolution into a depthwise convolution (one filter per input channel, spatial-only) followed by a pointwise 1×1 convolution (combining channels), drastically reducing parameters/FLOPs compared to a standard convolution, at a small accuracy cost — key for efficient/mobile architectures.

Q83. What is a Squeeze-and-Excitation (SE) block, and what does "csSE" add for segmentation? An SE block adaptively recalibrates channel-wise feature responses: it "squeezes" spatial information into a per-channel descriptor (global average pooling), then "excites" it through a small MLP + sigmoid to produce per-channel attention weights that rescale the original feature map — letting the network emphasize informative channels. csSE (concurrent spatial and channel Squeeze-and-Excitation) extends this with a parallel spatial squeeze-excitation branch (a 1×1 conv + sigmoid producing a per-pixel attention map), and combines both branches (typically via max or sum), which is especially effective in medical/fine-grained segmentation where both "which channels" and "which pixels" matter.

Q84. What is EfficientNet's compound scaling? Rather than arbitrarily scaling depth, width, or input resolution independently, EfficientNet scales all three jointly using a fixed compound coefficient, found via neural architecture search to balance accuracy and efficiency more effectively than scaling any single dimension alone.

Q85. Batch Normalization's role specifically in CNNs — where is it applied? Typically after the convolution and before the activation function, normalizing per-channel across the batch and all spatial locations (N×H×W), which stabilizes training and allows larger learning rates in deep conv stacks.

---

9. Sequence Models

Q86. What limitation of vanilla RNNs motivated LSTM/GRU? Vanilla RNNs suffer from vanishing (and sometimes exploding) gradients over long sequences because the same weight matrix is repeatedly multiplied through backpropagation-through-time, making it hard to learn long-range dependencies.

Q87. Explain the LSTM cell and its gates. LSTM maintains a cell state (long-term memory) alongside a hidden state, regulated by three gates: forget gate (what to discard from cell state), input gate (what new information to add), and output gate (what to expose as the hidden state). The additive cell-state update (rather than repeated multiplication) helps gradients flow over long sequences.

Q88. Explain GRU and how it differs from LSTM. GRU merges the cell and hidden states into one and uses two gates: a reset gate (how much past information to forget when computing the candidate state) and an update gate (how much to blend the candidate with the previous state). It has fewer parameters than LSTM and often performs comparably while being faster to train.

Q89. What is Backpropagation Through Time (BPTT)? It's backpropagation applied to an RNN "unrolled" across time steps, treating each time step as a layer and accumulating gradients with

respect to shared weights across all time steps — computationally expensive and prone to vanishing/exploding gradients for long sequences.

Q90. What is the seq2seq (encoder-decoder) architecture? An encoder RNN/LSTM compresses an input sequence into a fixed-size context vector (final hidden state); a decoder RNN generates the output sequence conditioned on that context, used for tasks like machine translation. The fixed-size bottleneck was a key limitation that attention mechanisms were introduced to solve.

Q91. What problem did Bahdanau (additive) attention solve in seq2seq models? Instead of forcing the encoder to compress the entire input into one fixed vector, attention lets the decoder, at each output step, compute a weighted combination over all encoder hidden states (weights learned based on relevance to the current decoding step) — directly addressing the information bottleneck and improving performance on long sequences.

---

10. Transformers & Attention

Q92. Write the scaled dot-product attention formula and explain each term. Attention(Q,K,V) = softmax($QK^T/\sqrt{d\_k}$) V. Q (queries), K (keys), V (values) are learned linear projections of the input. QK^T computes similarity scores between queries and keys; dividing by $\sqrt{d\_k}$ prevents large dot products from pushing softmax into regions with vanishing gradients; softmax converts scores to weights; the weighted sum of V produces the output.

Q93. What is multi-head attention, and why use multiple heads instead of one? Instead of a single attention function, the input is linearly projected into h separate (Q,K,V) sets ("heads"), each attending independently in a lower-dimensional subspace, and their outputs are concatenated and linearly projected back. This lets the model jointly attend to information from different representation subspaces (e.g., syntactic vs semantic relationships) simultaneously.

Q94. Explain self-attention vs cross-attention. Self-attention: Q, K, and V all come from the same sequence — each token attends to other tokens within that same sequence (used in encoder layers, and in decoder layers over previously generated tokens). Cross-attention: Q comes from one sequence (e.g., the decoder) while K and V come from a different sequence (e.g., the encoder's output) — this is how the decoder "looks at" the encoded input, and is also central to how text conditions image generation in diffusion models or how CLIP-like grounding works.

Q95. Where exactly does cross-attention appear in the original Transformer decoder, and what does it do? Each decoder block has: (1) masked self-attention over previously generated output tokens, (2) cross-attention where queries come from the decoder's current representation and keys/values come from the encoder's final output, (3) a feed-forward layer. The cross-attention step is what lets the decoder condition its next-token prediction on the full source sequence.

Q96. Why is positional encoding necessary in Transformers? Self-attention is permutation-invariant — it has no inherent notion of token order (unlike RNNs, which process sequentially). Positional encodings inject order information by adding a position-dependent vector to each token embedding before the first layer.

Q97. Explain sinusoidal positional encoding. Each position pos and dimension i gets PE(pos,2i) = sin(pos/10000^(2i/d)), PE(pos,2i+1) = cos(pos/10000^(2i/d)). Using different frequencies per dimension lets the model learn to attend by relative position (since PE(pos+k) can be expressed as a

linear function of PE(pos)), and it generalizes to sequence lengths not seen during training.

Q98. What is the feed-forward sublayer in a Transformer block, and what's its role? A position-wise two-layer MLP (typically expanding to 4× the model dimension then projecting back) applied identically and independently to each token's representation after attention. It adds non-linear transformation capacity per-token, since attention itself is primarily a (linear) mixing/routing mechanism.

Q99. Why do Transformer blocks use residual connections around both attention and FFN sublayers? Same rationale as ResNet: residual connections let gradients flow directly through the network, which is essential for training the very deep stacks used in modern Transformers, and let each sublayer learn a refinement/delta rather than a full transformation from scratch.

Q100. What is the computational complexity of self-attention, and why does it matter? $O(n^2 \cdot d)$ in sequence length n (every token attends to every other token), which becomes a bottleneck for long sequences — motivating efficient-attention variants (sparse attention, linear attention, FlashAttention for memory-efficient exact computation, sliding-window attention, etc.).

Q101. What is masked self-attention, and why is it needed in decoders? It applies a mask (setting disallowed positions' attention scores to $-\infty$ before softmax) so that a token can only attend to itself and earlier tokens, not future ones — necessary for autoregressive generation, ensuring training is consistent with the left-to-right inference process.

Q102. Encoder-only vs decoder-only vs encoder-decoder Transformers — give an example of each. Encoder-only (bidirectional self-attention, good for understanding tasks): BERT. Decoder-only (causal/masked self-attention, good for generation): GPT family. Encoder-decoder (encoder processes input bidirectionally, decoder generates output with cross-attention to encoder output): original Transformer, T5, machine translation models.

---

11. Transformer-Based Models

Q103. Explain BERT's pretraining objectives. Masked Language Modeling (MLM): randomly mask ~15% of input tokens and train the model to predict them using bidirectional context. Next Sentence Prediction (NSP): predict whether two sentences are consecutive in the original text (later work like RoBERTa found NSP largely unnecessary and dropped it, instead training longer with more data and dynamic masking).

Q104. Why is BERT bidirectional while GPT is unidirectional, and what does that imply about their use cases? BERT's encoder-only self-attention has no causal mask, so every token can attend to tokens both before and after it — ideal for understanding tasks (classification, NER, QA) where full context is available. GPT uses masked (causal) self-attention so each token only sees prior tokens, matching the autoregressive generation setting where future tokens aren't available at inference time.

Q105. What is the [CLS] token in BERT used for? A special token prepended to every input sequence whose final hidden representation is used as an aggregate sequence-level representation, typically fed into a classification head for tasks like sentiment analysis or natural language inference.

Q106. Explain the Vision Transformer (ViT) architecture. An image is split into fixed-size patches (e.g., 16×16), each patch is flattened and linearly projected into an embedding ("patch embedding"), a learnable [CLS] token and positional embeddings are added, and the resulting sequence is fed through a standard Transformer encoder — treating an image essentially as a "sentence" of patches.

Q107. Why does ViT typically need more data or specific training tricks to match CNN performance from scratch? CNNs have built-in inductive biases (locality, translation equivariance via weight sharing) that help them generalize well even on smaller datasets. ViT lacks these biases (attention is global from layer one), so it needs either large-scale pretraining data (e.g., JFT-300M) or strong data augmentation/distillation (as in DeiT) to learn useful spatial priors from data alone.

Q108. What is DINO, and how does it train without labels? DINO (self-**DI**stillation with **NO** labels) trains a student and a teacher network (both same architecture, teacher is an exponential moving average of the student) to produce matching output distributions for different augmented views (global and local crops) of the same image, using only a cross-entropy-style loss between student and teacher outputs — no labels needed. Centering and sharpening the teacher's output distribution prevents collapse to a trivial constant output.

Q109. What emergent property did DINO discover in ViT features, and why is it notable? Self-supervised ViT features trained with DINO exhibit attention maps that naturally segment objects from background (semantic segmentation-like structure) without ever being trained on segmentation labels — an emergent property not observed to the same degree in supervised ViTs, suggesting self-supervision encourages more semantically meaningful representations.

Q110. What does DINOv2 add over DINO? DINOv2 scales up data (curated, deduplicated large-scale datasets), combines the DINO self-distillation objective with a masked-image-modeling loss (iBOT-style) and additional stabilization techniques (e.g., KoLeo regularization, better teacher/student setup), producing general-purpose visual features that transfer well across many downstream tasks (classification, segmentation, depth estimation) without fine-tuning.

Q111. Explain CLIP's training objective. CLIP jointly trains an image encoder and a text encoder using contrastive learning: for a batch of N (image, caption) pairs, it computes similarity between every image and every text embedding, and optimizes (via a symmetric InfoNCE loss) so that matching pairs have high cosine similarity while all other (mismatched) pairs in the batch have low similarity.

Q112. How does CLIP perform zero-shot image classification? Class names are embedded into text prompts (e.g., "a photo of a {class}"), encoded via the text encoder; an image is encoded via the image encoder; the predicted class is whichever text embedding has the highest cosine similarity to the image embedding — no task-specific fine-tuning or labeled training images needed.

Q113. What does "CLIP-grounded" mean in the context of grounding vision features with language (e.g., for segmentation)? It refers to using CLIP's joint image-text embedding space to inject language/semantic information into visual features — e.g., computing similarity between CLIP text embeddings of class/attribute names and dense per-pixel or per-patch visual features (from a backbone like DINOv2) to produce segmentation masks guided by natural-language descriptions, rather than relying solely on fixed label-indexed classifier heads.

Q114. Why combine a DINOv2 backbone with CLIP-style text grounding rather than using

CLIP's own visual encoder for dense prediction? CLIP's image encoder is trained for global image-text alignment and its dense/patch-level features are comparatively weaker for fine-grained spatial tasks. DINOv2 produces strong dense, spatially-precise self-supervised features (good for segmentation-style tasks), so combining DINOv2's spatial features with CLIP's language grounding leverages the strengths of each: DINOv2 for "where," CLIP-derived embeddings for "what."

Q115. What is knowledge distillation, and how does it relate to DINO's teacher-student setup? Knowledge distillation trains a (typically smaller or auxiliary) student model to match the output distribution of a teacher model, transferring learned knowledge without needing ground-truth labels. In DINO, the "teacher" is an EMA of the student itself (self-distillation) rather than a separately pretrained, larger, fixed model — an unusual but effective variant.

---

12. Parameter-Efficient Fine-Tuning (LoRA & friends)

Q116. Explain LoRA (Low-Rank Adaptation). Instead of fine-tuning all weights W of a pretrained model, LoRA freezes W and learns a low-rank update $\Delta W = BA$, where $B \in \mathbb{R}^{(d\times r)}$ and $A \in \mathbb{R}^{(r\times k)}$ with rank $r \ll \min(d,k)$. The forward pass becomes $h = Wx + BAx$. Only A and B (a tiny fraction of the original parameter count) are trained, drastically reducing memory/compute for fine-tuning while matching full fine-tuning performance on many tasks.

Q117. Why does a low-rank assumption work for fine-tuning large pretrained models? Empirically (and motivated by the "intrinsic dimensionality" hypothesis), the weight updates needed to adapt a large pretrained model to a downstream task tend to have a low "intrinsic rank" — i.e., most of the useful update can be captured in a small number of directions, even though the full weight matrix is high-dimensional.

Q118. How is LoRA initialized, and why does that matter? A is typically initialized with small random values (e.g., Gaussian) and B is initialized to zero, so that $BA = 0$ at the start of training — meaning the model behaves identically to the original pretrained model before any fine-tuning steps, ensuring a stable starting point.

Q119. What is the effect of LoRA's rank r and scaling factor α? Higher rank r increases the update's expressive capacity (more trainable parameters) at the cost of memory/compute; very low rank may underfit the target task. The scaling factor α/r scales the magnitude of the update ($h = Wx + (\alpha/r)BAx$), controlling how strongly the LoRA update influences the output relative to the frozen weights — tuned similarly to a learning-rate-like hyperparameter.

Q120. What is QLoRA, and what problem does it solve? QLoRA fine-tunes LoRA adapters on top of a base model whose frozen weights are quantized to a low-precision format (e.g., 4-bit NormalFloat), plus techniques like double quantization and paged optimizers — enabling fine-tuning of very large models (tens of billions of parameters) on a single consumer/prosumer GPU by drastically reducing memory needed for the frozen backbone.

Q121. Compare LoRA to other PEFT methods: adapters and prefix-tuning. Adapters insert small bottleneck MLP modules between existing layers (adding inference latency since they're in the forward path). Prefix/prompt-tuning prepends trainable "virtual tokens" to the input/hidden states at each layer, leaving the backbone untouched. LoRA modifies weight matrices via a low-rank additive term that can be merged back into the original weights after training, adding zero

extra inference latency (unlike adapters) — a key practical advantage.

Q122. Which weight matrices are typically targeted by LoRA in a Transformer, and why? Most commonly the attention projection matrices (particularly the query and value projections, W_q and W_v), since empirically adapting these tends to give the best performance-per-parameter tradeoff; some setups also apply LoRA to the feed-forward layers for further gains at added parameter cost.

---

13. Generative Models

Q123. What is an autoencoder, and what is it used for? An unsupervised network with an encoder (compresses input x to a lower-dimensional latent code z) and a decoder (reconstructs $\hat{x}$ from z), trained to minimize reconstruction error. Used for dimensionality reduction, denoising, anomaly detection, and as a building block for representation learning.

Q124. How does a Variational Autoencoder (VAE) differ from a standard autoencoder? A VAE's encoder outputs parameters (mean μ, variance σ^2) of a distribution over the latent space rather than a single deterministic point, and a sample z is drawn from this distribution (via the reparameterization trick) before decoding. This makes the latent space continuous and structured, enabling generation of new samples by sampling z from the prior.

Q125. What is the VAE loss function (ELBO), and what does each term do? The Evidence Lower Bound: $L = E[\log p(x|z)] - KL(q(z|x) \,||\, p(z))$. The first term is a reconstruction loss (how well the decoder reconstructs x from the sampled z). The second is a KL-divergence regularizer that pulls the learned latent distribution q(z|x) toward a prior (usually standard normal), keeping the latent space smooth and well-structured for sampling/generation.

Q126. Explain the reparameterization trick and why it's needed. Since z ~~$\sim N(\mu,\sigma^2)$ involves a stochastic sampling operation, gradients can't flow through it directly via backpropagation. The trick rewrites $z = \mu + \sigma \odot \epsilon$ where ϵ~~ $\sim N(0,I)$ is sampled independently — moving randomness to an input that doesn't require gradients, making μ and σ (both deterministic functions of the encoder) differentiable with respect to the loss.

Q127. Explain the GAN framework — generator, discriminator, and the minimax objective. A generator G maps noise z to fake samples G(z); a discriminator D tries to distinguish real samples from fake ones. They're trained adversarially: $\min_G \max_D E[\log D(x)] + E[\log(1-D(G(z)))]$ — D tries to maximize its classification accuracy, G tries to fool D into thinking its outputs are real.

Q128. What is mode collapse in GANs, and why does it happen? The generator learns to produce only a limited variety of outputs (or even a single output) that reliably fools the discriminator, rather than capturing the full diversity of the real data distribution — often because the generator finds a "safe" mode that consistently scores well, and gradient-based training doesn't inherently encourage output diversity.

Q129. Why are GANs notoriously hard to train? The adversarial minimax game can be unstable (oscillating rather than converging), gradients can vanish if the discriminator becomes too strong too quickly, and there's no single scalar loss that reliably indicates sample quality/progress the way a supervised loss does — necessitating tricks like Wasserstein loss, gradient penalties, spectral

normalization, and careful learning rate balancing.

Q130. VAE vs GAN — key tradeoffs? VAEs have a well-defined likelihood-based training objective, stable training, and a structured latent space, but tend to produce blurrier samples (due to the reconstruction loss averaging over plausible outputs). GANs produce sharper, more realistic samples but are harder/less stable to train and lack an explicit likelihood or straightforward way to evaluate convergence.

Q131. What is the core idea behind diffusion models? A forward process gradually adds Gaussian noise to data over many steps until it becomes pure noise; a neural network is trained to reverse this process — predicting (and removing) the noise at each step — so that starting from random noise and iteratively denoising produces a realistic sample.

Q132. What does a diffusion model's training objective actually optimize? In practice (DDPM), the network is trained to predict the noise ε added at a randomly sampled timestep t, minimizing a simple mean-squared-error loss between the true noise and predicted noise — a simplified, reweighted version of the variational lower bound on the data likelihood.

Q133. Why do diffusion models generally produce higher-quality/more diverse samples than GANs, at what cost? Their iterative denoising process and likelihood-based training avoid the adversarial instability and mode collapse issues of GANs, generally yielding higher fidelity and diversity. The cost is inference speed — generating a sample requires many sequential denoising steps (though distillation and faster samplers like DDIM have significantly reduced this gap).

Q134. How does classifier-free guidance work in text-to-image diffusion models? During training, the conditioning (e.g., text prompt) is randomly dropped so the model learns both a conditional and unconditional denoiser. At inference, the final noise prediction is extrapolated as $\varepsilon = \varepsilon_{uncond} + w \cdot (\varepsilon_{cond} - \varepsilon_{uncond})$, amplifying the influence of the conditioning signal (higher guidance weight w = stronger prompt adherence, at some cost to diversity).

---

14. Self-Supervised & Contrastive Learning

Q135. What problem does self-supervised learning address? Labeled data is expensive and limited; self-supervised learning creates supervisory signals from the data itself (e.g., predicting masked parts, matching augmented views) so models can learn useful representations from large amounts of unlabeled data, which can then be fine-tuned or used directly for downstream tasks.

Q136. Explain contrastive learning and the InfoNCE loss. Contrastive learning trains representations so that "positive pairs" (e.g., two augmented views of the same image) are pulled close together in embedding space while "negative pairs" (views of different images) are pushed apart. InfoNCE loss (used in SimCLR, CLIP, MoCo) is essentially a softmax classification over one positive among many negatives: $-\log[\exp(\text{sim}(z_i,z_j)/\tau) / \Sigma_k \exp(\text{sim}(z_i,z_k)/\tau)]$, where τ is a temperature hyperparameter.

Q137. What is the role of temperature (τ) in contrastive losses? It controls how sharply the similarity scores are scaled before softmax — a low temperature makes the loss focus heavily on the hardest negatives (sharper distribution), while a high temperature treats all negatives more uniformly (softer distribution); it's an important tunable hyperparameter for contrastive learning performance.

Q138. What are "positive" and "negative" pairs in SimCLR, and why does batch size matter so much? Positives are two independently augmented views of the same image; negatives are all other images in the current batch. Larger batches provide more negative examples per training step, generally improving the quality of learned representations, which is why SimCLR benefits significantly from large batch sizes (or a memory bank/queue, as in MoCo).

Q139. How does MoCo (Momentum Contrast) avoid needing very large batch sizes for enough negatives? It maintains a large queue of negative embeddings computed by a separate "momentum encoder" (an EMA of the main encoder, updated slowly for consistency), decoupling the number of available negatives from the training batch size — enabling effective contrastive learning without requiring huge batches.

Q140. Explain masked image modeling (e.g., MAE) as a self-supervised objective. Similar to BERT's MLM but for images: a large fraction of image patches (e.g., 75%) are masked out, and an encoder-decoder is trained to reconstruct the missing pixel values from the visible patches only. The encoder only processes visible patches (efficient), and a lightweight decoder reconstructs the full image, learning strong representations useful for downstream fine-tuning.

Q141. Why does DINO avoid using explicit negative pairs (unlike SimCLR/MoCo), and how does it avoid representational collapse? DINO only uses positive pairs (different augmented views matched between student and teacher), avoiding the need to mine or construct negatives. Collapse (all outputs becoming constant/identical) is prevented via centering (subtracting a running mean from teacher outputs) and sharpening (low-temperature softmax on the teacher), which together avoid a trivial uniform or one-hot solution.

---

15. Segmentation & Detection

Q142. Semantic vs instance vs panoptic segmentation — what's the difference? Semantic segmentation assigns a class label to every pixel without distinguishing individual object instances (all "cars" get one label). Instance segmentation additionally separates individual object instances (car #1 vs car #2). Panoptic segmentation unifies both: instance-level masks for "things" (countable objects) and semantic labels for "stuff" (background regions like sky, road).

Q143. Explain the U-Net architecture. U-Net has a contracting encoder path (downsampling, capturing context) and an expanding decoder path (upsampling, for precise localization), with skip connections directly linking corresponding encoder and decoder resolutions — passing high-resolution spatial detail that would otherwise be lost through downsampling, crucial for pixel-accurate segmentation.

Q144. Why are skip connections especially important in segmentation architectures like U-Net? Downsampling in the encoder loses fine spatial detail needed for precise pixel-level boundaries. Skip connections reinject that high-resolution, low-level detail directly into the decoder at matching scales, letting the network combine coarse semantic context (from deep layers) with fine spatial precision (from shallow layers).

Q145. Define IoU (Intersection over Union) and Dice coefficient — how do they differ? IoU = |Prediction ∩ GroundTruth| / |Prediction ∪ GroundTruth|. Dice = 2|Prediction ∩ GroundTruth| / (|Prediction| + |GroundTruth|). They're related (Dice = 2·IoU/(1+IoU)) and both measure overlap, but

Dice weights the intersection more heavily and is more sensitive/forgiving for small objects — commonly preferred in medical image segmentation.

Q146. Why is Dice loss (or a combination with cross-entropy) often preferred over plain cross-entropy for segmentation, especially with class imbalance? Pixel-wise cross-entropy treats all pixels independently and can be dominated by the majority class (e.g., background) in imbalanced segmentation tasks, leading the model to trivially predict the majority class well while ignoring small foreground structures. Dice loss directly optimizes overlap and is inherently more robust to class imbalance; combining Dice + cross-entropy often gives more stable gradients than Dice alone.

Q147. Trace the evolution: R-CNN → Fast R-CNN → Faster R-CNN. R-CNN: runs a CNN independently on ~2000 externally-generated region proposals (selective search) per image — very slow. Fast R-CNN: runs the CNN once on the whole image, then uses RoI pooling to extract per-proposal features from the shared feature map — much faster, but region proposals are still external/slow. Faster R-CNN: replaces external proposals with a learned Region Proposal Network (RPN) sharing the same backbone features, making the entire pipeline end-to-end trainable and much faster.

Q148. How does YOLO differ fundamentally from the R-CNN family? YOLO (You Only Look Once) treats detection as a single regression problem: it divides the image into a grid and, in one forward pass, directly predicts bounding boxes, objectness scores, and class probabilities per grid cell — a one-stage detector, versus R-CNN's two-stage propose-then-classify approach. This makes YOLO much faster (real-time capable) at some historical cost to accuracy on small/overlapping objects, though modern YOLO versions have substantially closed that gap.

Q149. What is Non-Maximum Suppression (NMS), and why is it needed? Detectors typically produce many overlapping candidate boxes for the same object. NMS keeps the highest-confidence box and suppresses (removes) other boxes that overlap it above an IoU threshold, reducing redundant detections to one box per actual object.

Q150. What is focal loss, and what problem in detection does it address? Focal loss down-weights the loss contribution of well-classified (easy) examples via a factor $(1-p_t)^\gamma$ multiplying the standard cross-entropy, focusing training on hard, misclassified examples. It was introduced to address the extreme foreground-background class imbalance in one-stage detectors, where the vast majority of candidate locations are easy negatives that would otherwise dominate the loss.

Q151. What is anchor-based vs anchor-free detection? Anchor-based methods (Faster R-CNN, original YOLO/SSD) predefine a set of reference boxes (anchors) of various scales/aspect ratios at each location and predict offsets/classes relative to them. Anchor-free methods (e.g., FCOS, CenterNet) directly predict object centers/keypoints and box dimensions without predefined anchors, simplifying the design and removing anchor-related hyperparameters.

Q152. How would you apply SE/csSE-style attention blocks in a segmentation network, and why might they help? Inserting csSE blocks after encoder/decoder conv blocks lets the network adaptively reweight both channels (which feature types matter) and spatial locations (which pixels matter) before passing features forward — useful for emphasizing thin/small structures (e.g., fine anatomical boundaries) that plain convolutions might underweight relative to larger, more common structures.

16. Reinforcement Learning

Q153. Define a Markov Decision Process (MDP). An MDP is defined by (S, A, P, R, γ): states S, actions A, transition probabilities P(s'|s,a), reward function R(s,a,s'), and discount factor $\gamma \in [0,1)$. The Markov property means the next state/reward depends only on the current state and action, not the full history.

Q154. What is the difference between a value function and a Q-function? V(s) is the expected cumulative discounted reward starting from state s (following a given policy). Q(s,a) is the expected cumulative discounted reward starting from state s, taking action a, then following the policy thereafter. V(s) = max_a Q(s,a) under an optimal policy.

Q155. State the Bellman optimality equation for Q-learning. $Q^*(s,a) = E[r + \gamma \cdot \max_{a'} Q^*(s',a')]$. It expresses the optimal Q-value as the immediate reward plus the discounted value of acting optimally from the next state onward — the basis for the Q-learning update rule: $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \cdot \max_{a'} Q(s',a') - Q(s,a)]$.

Q156. Why is Q-learning called "off-policy"? It learns the value of the optimal policy (using max_a' Q(s',a') in its update) regardless of which action was actually taken by the behavior policy that generated the experience (e.g., an ε-greedy exploration policy) — the policy being learned about differs from the policy generating the data.

Q157. What is the exploration-exploitation tradeoff, and how does ε-greedy address it? The agent must balance exploiting known good actions (maximize immediate expected reward) against exploring less-tried actions (to discover potentially better strategies and avoid getting stuck in a suboptimal policy). ε-greedy chooses the best-known action with probability $1-\varepsilon$ and a random action with probability ε, with ε often annealed over time.

Q158. What is the policy gradient theorem, in essence? It gives a way to compute the gradient of expected return with respect to policy parameters θ without needing to differentiate through the (often unknown/non-differentiable) environment dynamics: $\nabla\theta\, J(\theta) = E[\nabla\theta \log \pi_\theta(a|s) \cdot Q^{\pi}(s,a)]$ — increasing the probability of actions proportionally to how much better-than-expected they turned out.

Q159. Why do policy gradient methods typically subtract a baseline (e.g., using advantage instead of raw return)? Using raw returns as the scaling factor gives high-variance gradient estimates. Subtracting a baseline (commonly the value function V(s), giving the advantage A(s,a) = Q(s,a) − V(s)) reduces variance without introducing bias (as long as the baseline doesn't depend on the action), leading to more stable training.

Q160. Explain Actor-Critic methods. The "actor" is the policy network, updated via the policy gradient. The "critic" is a value/advantage estimator that provides a lower-variance learning signal to the actor than raw Monte Carlo returns would, and is itself trained via TD-error minimization. This combines the benefits of policy-based (direct, works with continuous actions) and value-based (lower variance, bootstrap) methods.

Q161. Explain PPO (Proximal Policy Optimization) and its clipped surrogate objective. PPO improves training stability over vanilla policy gradients by limiting how far a policy update can move from the previous policy in a single step. Its clipped objective: $L(\theta) = E[\min(r_t(\theta)\cdot \hat{A}_t,$

clip(r_t(θ), 1−ε, 1+ε)·Â_t)], where r_t(θ) = π_θ(a|s)/π_θ_old(a|s) is the probability ratio and Â_t is the estimated advantage. Clipping discourages excessively large policy updates that could destabilize training, without requiring the more complex second-order trust-region computation of TRPO.

Q162. Why is PPO popular in practice compared to earlier methods like TRPO or vanilla policy gradient? It achieves similar stability benefits to TRPO's trust-region constraint using a much simpler first-order optimization (just clipping in the loss, no constrained optimization or Fisher information matrix computation), making it easier to implement, more compute-efficient, and less sensitive to hyperparameters — while typically matching or exceeding TRPO's sample efficiency and stability in practice.

Q163. On-policy vs off-policy — what's the tradeoff, and where does PPO fall? On-policy methods (PPO, vanilla policy gradient) can only learn from data generated by the current policy, requiring fresh data after every update (sample-inefficient but generally simpler and more stable). Off-policy methods (Q-learning, DDPG, SAC) can reuse past experience via a replay buffer (more sample-efficient) but are often less stable and harder to tune. PPO is on-policy, though it does reuse each batch of collected rollouts for a few epochs of minibatch updates before discarding it.

Q164. What is reward shaping, and what risk does it carry? Reward shaping modifies/augments the raw environment reward to provide denser or more informative learning signal (e.g., partial credit for progress toward a goal). The risk is that a poorly designed shaped reward can lead the agent to exploit unintended shortcuts ("reward hacking") that maximize the shaped reward without actually achieving the intended task/behavior.

Q165. What is the credit assignment problem in RL? Determining which past actions were actually responsible for a later reward, especially when rewards are sparse and delayed — an action taken many steps before a reward may be hard to correctly credit (or blame), which is part of why discounting, eligibility traces, and advantage estimation techniques exist.

---

17. Training & Optimization Practicalities

Q166. What is learning rate warmup, and why is it commonly used (especially for Transformers)? Warmup linearly increases the learning rate from a small value up to the target LR over the first several steps/epochs before applying the main schedule (e.g., cosine decay). It prevents large, destabilizing updates early in training when weights (and, in adaptive optimizers, second-moment estimates) are poorly initialized/estimated.

Q167. Explain cosine annealing / cosine decay learning rate schedules. The learning rate follows a cosine curve, starting high and smoothly decreasing to a small value (often near zero) by the end of training, sometimes with restarts (cosine annealing with warm restarts). This tends to produce better final convergence than a fixed or step-decayed learning rate by allowing large exploratory steps early and fine-grained refinement later.

Q168. What is gradient clipping, and when is it necessary? It rescales gradients if their norm exceeds a threshold (clip_grad_norm), preventing exploding gradients from causing destabilizing large parameter updates — particularly important in RNNs/LSTMs and in RL/Transformer training where occasional large gradient spikes are common.

Q169. Explain mixed-precision training and why loss scaling is needed. Mixed precision uses FP16 (or bfloat16) for most computations (faster, less memory) while keeping a master copy of weights in FP32 for stable updates. With FP16, gradient values can underflow to zero (FP16's limited exponent range), so loss scaling multiplies the loss by a large factor before backpropagation (then unscales gradients before the optimizer step) to keep small gradient values representable.

Q170. What is the effect of batch size on generalization, and what is the "large batch generalization gap"? Very large batch sizes tend to converge to sharper minima in the loss landscape, which have been empirically associated with worse generalization compared to the flatter minima found with smaller batches/more gradient noise — though this gap can often be mitigated with learning rate scaling (e.g., linear scaling rule) and warmup.

Q171. What is transfer learning, and what are common strategies for fine-tuning a pretrained model? Reusing a model pretrained on a large source dataset/task as a starting point for a related target task. Strategies: (1) freeze the backbone and train only a new head (feature extraction — good for small target datasets), (2) fine-tune the whole network with a small learning rate (good with more target data), (3) gradual unfreezing / discriminative learning rates (unfreeze layers progressively, often with smaller LR for earlier/more general layers).

Q172. What is data augmentation, and give examples for images vs text. Artificially expanding training data by applying label-preserving transformations, improving generalization/robustness. Images: random crop, flip, rotation, color jitter, Cutout, MixUp, CutMix. Text: synonym replacement, back-translation, random token masking/deletion, paraphrasing.

Q173. Explain MixUp and CutMix. MixUp creates training examples by linearly interpolating both the pixels and labels of two images: $\tilde{x} = \lambda x_i + (1-\lambda)x_j$, $\tilde{y} = \lambda y_i + (1-\lambda)y_j$. CutMix instead pastes a rectangular patch from one image onto another, with the label mixed in proportion to the patch's area — both encourage smoother decision boundaries and better calibration/generalization.

Q174. What is early stopping, and how is it implemented? Halting training once validation performance stops improving (or starts degrading) for a set number of epochs ("patience"), to prevent overfitting to the training set — the model checkpoint with the best validation metric is typically retained rather than the final one.

Q175. Why can a very high learning rate cause training loss to diverge (NaN), and what's the fix? Excessively large updates overshoot minima and can push weights (and subsequently activations/gradients) to extreme values that overflow to NaN/Inf, especially compounded across many layers. Fixes: lower the learning rate, add warmup, use gradient clipping, check for numerical issues (e.g., log(0), division by zero) in the loss computation, and verify data/label preprocessing.

Q176. Compare SGD with momentum vs Adam — when might you prefer one over the other? SGD+momentum often generalizes slightly better and is the standard for many vision tasks (e.g., ResNet training) given a well-tuned learning rate schedule, but requires more careful LR tuning. Adam (adaptive per-parameter learning rates) converges faster and is more robust to hyperparameter choices out of the box, and is the default for Transformers/NLP — though it can sometimes generalize slightly worse than well-tuned SGD, motivating variants like AdamW (decoupled weight decay).

Q177. What is weight decay, and how does AdamW's decoupled weight decay differ from standard L2 regularization in Adam? Weight decay directly shrinks weights toward zero at each

step ($\theta \leftarrow \theta - \eta \cdot \lambda \cdot \theta$), separate from the gradient-based update. In vanilla Adam, adding an L2 penalty to the loss interacts poorly with Adam's adaptive per-parameter scaling (the effective decay ends up inversely scaled by the adaptive learning rate). AdamW decouples weight decay from the gradient-adaptive update entirely, applying it as a separate step — shown empirically to improve generalization.

Q178. What is distributed data-parallel training, and how does gradient synchronization work? The model is replicated across multiple devices/GPUs, each processing a different shard of the mini-batch in parallel; after the backward pass, gradients are averaged across all replicas (typically via an all-reduce operation) before the optimizer step, so all replicas stay synchronized with identical weights.

Q179. What is gradient checkpointing (activation checkpointing), and what tradeoff does it make? Instead of storing all intermediate activations for the backward pass, it stores only a subset (checkpoints) and recomputes the rest on-the-fly during backpropagation — trading increased compute (extra forward passes) for significantly reduced memory usage, enabling training of larger models/batch sizes than would otherwise fit in memory.

---

18. Debugging & Applied Engineering Questions

Q180. Your training loss isn't decreasing at all from the start. What do you check? Learning rate (too high causing divergence, or too low causing near-zero progress), whether gradients are actually flowing (check for frozen layers, detached tensors, or a broken loss computation), correct loss function for the task, label/data preprocessing correctness (e.g., normalization, off-by-one label indexing), and whether the model can overfit a tiny subset of data (a standard sanity check — if it can't, there's a bug).

Q181. Training loss decreases but validation loss increases early — what's happening and what would you try? Classic overfitting. Try: more/better data augmentation, stronger regularization (dropout, weight decay), reducing model capacity, early stopping, or acquiring more training data; also verify the train/val split doesn't have leakage in the wrong direction (i.e., that validation truly is held out).

Q182. Your model achieves near-perfect training accuracy but performs poorly at deployment/on new data — what would you investigate? Check for data leakage (features that leak the label, or improper temporal/entity split allowing information from the "future" or same entity to appear in both train and test), distribution shift between training data and real-world deployment data, and whether evaluation metrics/pipeline match between offline validation and production.

Q183. How would you debug an RNN/Transformer producing repetitive or degenerate output during generation? Check the decoding strategy (greedy decoding is prone to repetition; try beam search with repetition penalty, or sampling methods like top-k/top-p/temperature sampling), verify the model was trained long enough/on diverse enough data, and check for teacher-forcing exposure bias (train-time next-token-given-ground-truth vs inference-time next-token-given-own-predictions mismatch).

Q184. How do you decide between a simpler classical ML model and a deep learning model

for a given problem? Consider dataset size (deep learning typically needs more data to outperform classical methods), interpretability requirements (trees/linear models are more interpretable), the nature of the data (tabular data often favors gradient-boosted trees over deep nets; unstructured data like images/text/audio strongly favors deep learning), latency/deployment constraints, and available compute/engineering resources.

Q185. How would you diagnose whether a segmentation model's poor performance is due to the loss function, architecture, or data? Run controlled ablations: swap only the loss function (e.g., cross-entropy vs Dice vs combined) while holding architecture/data fixed; visualize predictions vs ground truth on a handful of examples to spot systematic failure patterns (e.g., always missing small objects → possibly a class-imbalance/loss issue, or boundary blur → possibly needs skip connections/higher resolution); check dataset label quality and class distribution directly before assuming a modeling issue.

Q186. What steps would you take to reduce a model's inference latency for production deployment? Quantization (FP16/INT8), pruning, knowledge distillation into a smaller model, operator fusion/graph optimization (e.g., via ONNX/TensorRT), batching requests, caching, reducing input resolution/sequence length if acceptable, and choosing a more efficient architecture (e.g., MobileNet/EfficientNet over a heavier backbone) if the accuracy tradeoff is acceptable.

Q187. How do you evaluate whether a self-supervised pretrained model (e.g., DINOv2 features) is actually useful for your downstream task before fully fine-tuning? Linear probing: freeze the backbone, train only a lightweight linear (or shallow) head on top of the frozen features for the downstream task, and compare performance against training from scratch or against other backbones — a fast, standard way to assess representation quality without the cost of full fine-tuning.

This reference is intentionally dense — treat each answer as a starting point for deeper follow-up (derivations, code implementation, or paper references) depending on what you're preparing for.